

INTERVIEW GUIDE • EASY READ • FOR SATHISH

# Agentic AI / Semantic Solutions Architect

Same content as before, now in plain, simple language. Short sentences, hard words explained.  
Building AI agents that work over a company's knowledge graph, based on your real Takeda work.

FIT 82-85%

5 TOPICS

29 QUESTIONS

PLAIN ENGLISH

# How well you fit this job

Lead with your strong points. For the weak spot, say you use the ontology and let a partner build it.

|                                        |                                                                                                                                          |                |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>Agentic AI architecture</b>         | You use AutoGen in real work at Takeda, plus your agent-based CI/CD layer. You know planning loops and tool use.                         | <b>STRONG</b>  |
| <b>GraphRAG design</b>                 | You have built real GraphRAG. Mixing a graph with vector search is the thing that sets you apart from people who only know basic search. | <b>STRONG</b>  |
| <b>Context engineering</b>             | Your strong point. The 971-chunk pipeline, your chunking method, and how you test search quality.                                        | <b>STRONG</b>  |
| <b>MCP design</b>                      | You have a real, deployed MCP server with a proper safety boundary. This is the rarest skill on the list.                                | <b>STRONG</b>  |
| <b>Orchestration frameworks</b>        | You know LangChain, LlamaIndex, and AutoGen. LangGraph is the one you have not used yet, but it is close to LangChain.                   | <b>MOSTLY</b>  |
| <b>Semantic systems and ontologies</b> | GraphRAG gets you close. Deep ontology work (RDF, OWL, SPARQL) is the real stretch for you.                                              | <b>STRETCH</b> |

## How to handle the weak spot

The job post says you work with Semantic Data Architects to shape the ontology. That means a partner owns the ontology, not you. Your job is the agent side: you take what the graph gives you and make agents use it well. Do not pretend to be an ontology expert. Be the person who makes the graph useful to agents.

## One change to your pitch

This job is about quick prototypes (POC), not big production systems. Normally you lead with production results and big numbers. Here, lead with clear design thinking (MCP safety, GraphRAG search, context handling). Use your production background as proof that you know what the prototype will need later, not as the main point.

# Quick notes

|                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Agentic AI</b> <ul style="list-style-type: none"><li>• Planning styles: ReAct, Plan-and-Execute, Reflection</li><li>• An agent = an AI + tools + memory + a loop with a stop rule</li><li>• Safety: step limit, spend limit, allowed tools, checks</li><li>• Team of agents: planner, worker, checker</li><li>• Test the path it took, not just the final answer</li></ul> | <b>GraphRAG</b> <ul style="list-style-type: none"><li>• Vector search = find similar text. Graph = real links.</li><li>• Use both for multi-step and big-picture questions</li><li>• Build it: pull out things and links, then group them</li><li>• Narrow question = nearby graph. Broad question = summaries</li><li>• Every answer must point back to a real source</li></ul> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Context + MCP

- Chunk by meaning, then test the size
- Add labels: source, section, things, date
- Summarise long text, do not just cut it off
- MCP = a standard way to connect AI to tools and data
- The MCP server is the safety boundary

### Semantic systems

- Ontology = the types of things, their traits, and links
- SPARQL queries RDF facts. Cypher queries Neo4j graphs.
- A catalog is the agent's map of what data exists
- Make search easy: short paths, clear link names
- Auto-tag with NLP + AI, mapped to ontology terms

# A. Agentic AI Architecture

*A team of AI agents, the tools they use, and how they plan. Five of the six must-have skills start here. This is your home ground.*

## A1 How do you design a system with multiple AI agents? [INTERMEDIATE]

*Why they ask: They want to see if you think in terms of roles and teamwork, not just one big prompt.*

### How to answer

Think of it like a small team, not one person doing everything. I split the work across a few agents. One agent is the planner. It takes the big request and breaks it into small steps. Other agents are workers. Each worker does one job and uses one or two tools. One agent is the checker. It reviews the result before we send it back.

Why split it up? A small agent with one clear job is easier to control and easier to fix when something goes wrong. The cost is that more agents means more back-and-forth and more money spent. So I only split the work when the job really has separate parts. If the job is simple and straight, one agent with some tools is enough.

### Main points to make

- Treat it like a small team, not one big prompt
- A planner splits the work, workers do the jobs
- A checker reviews before the answer goes out
- More agents = more control but more cost
- Do not split up a simple job

### Likely follow-up

**They might ask:** When is one agent better than many?

**You say:** When the job is simple and uses one set of tools. More agents would just add cost and delay.

---

## A2 How do you make agents use their tools reliably? [INTERMEDIATE]

*Why they ask: Tools are where these systems often break. They want to know you can make it solid.*

### How to answer

Tools are how an agent does real things, like search a database or call an API. To make this reliable, I give each tool a clear shape: what it does, when to use it, and what inputs it needs. Before the agent runs a tool, I check the inputs are valid. If a tool fails, I send back a clear error the agent can understand, not a confusing crash. I also let it retry a few times, but with a hard limit so it cannot get stuck forever.

The biggest fix is to keep the list of tools small and clear. If two tools look similar, the agent gets confused and picks the wrong one. So I make sure each tool does one distinct thing. I also log every tool call, which helps me see what happened when something breaks.

### Main points to make

- Give each tool a clear shape and clear inputs
- Check the inputs before running the tool
- Return clear errors, allow limited retries
- Keep the tool list small so it does not get confused
- Log every tool call for debugging

### Likely follow-up

**They might ask:** What if it keeps picking the wrong tool?

**You say:** Usually two tools sound too alike. I make their descriptions clearer or combine them.

### A3 Explain ReAct, Plan-and-Execute, and Reflection. When do you use each? [EXPERT]

*Why they ask: They want to see depth on how agents plan, which is a must-have skill.*

#### How to answer

These are three ways an agent can think through a task. ReAct means think a little, act, see the result, then think again. It is flexible and good when you do not know the path ahead, but it can wander and costs more. Plan-and-Execute means make the full plan first, then carry it out step by step. It is faster and more predictable, but it does not adjust well if something surprising happens. Reflection means the agent checks its own answer and improves it. This raises quality but takes extra time.

In real life I mix them. I use Plan-and-Execute when the steps are clear, like a set onboarding flow. I use ReAct when the agent has to explore and find its way, like searching a knowledge graph. I add a Reflection step only when being correct matters more than being fast, like a risk report. The skill is picking the right style for the task, not having one favourite.

#### Main points to make

- ReAct: think, act, repeat. Flexible but wanders.
- Plan-and-Execute: plan first. Fast but less flexible.
- Reflection: the agent checks its own work. Better but slower.
- Mix them based on the task
- Clear steps = plan first. Exploring = ReAct.

#### Likely follow-up

**They might ask:** How do you stop a ReAct agent looping forever?

**You say:** I set a max number of steps and a spend limit. If it repeats itself, I stop it.

---

### A4 How do you stop an agent from going off the rails? [INTERMEDIATE]

*Why they ask: They want to know you can keep an agent safe, which matters a lot in a big regulated company.*

#### How to answer

I add safety layers. First, hard limits: a max number of steps, a spending limit, and a time limit, so nothing runs forever. Second, I only let the agent use an approved list of tools, and the MCP server blocks anything outside that list. Third, I check the inputs and outputs so bad actions are caught early. Fourth, for risky actions, a human approves before it happens.

I also log every step from the start. If something goes wrong, I can replay what the agent did and see exactly where it broke. You cannot keep an agent safe if you cannot see what it is doing.

#### Main points to make

- Hard limits: steps, spend, and time
- Only allow an approved list of tools
- Check inputs and outputs
- A human approves risky actions
- Log everything so you can replay it

#### Likely follow-up

**They might ask:** What about a bad instruction hidden in a document the agent reads?

**You say:** I treat anything the agent reads as untrusted. I keep instructions and data separate, and the MCP server still blocks any action it is not allowed to take.

## A5 AutoGen vs LangGraph vs CrewAI. Which would you pick? [INTERMEDIATE]

*Why they ask: They want to see your judgement on tools. This is also where LangGraph may come up.*

### How to answer

These are three tools for running a team of agents. AutoGen, which I use at work, is good when agents talk to each other to solve a problem, and it handles human approval well. CrewAI is simple and built around giving each agent a clear role, so you can get a team running quickly. LangGraph maps the whole flow as a chart of steps with shared memory, which gives you very precise control. It sits on top of LangChain.

I will be honest: I have hands-on experience with AutoGen, LangChain, and LlamaIndex, but not LangGraph yet. The good news is that the ideas are the same. The planner, worker, and checker setup I already build in AutoGen is the same idea LangGraph uses. So I am confident I can pick it up fast. The tool is the easy part once you understand how agents work together.

### Main points to make

- AutoGen: agents talk to each other. My main tool.
- CrewAI: simple, role-based teams. Quick to start.
- LangGraph: precise step-by-step control on LangChain.
- Honest: I have not used LangGraph yet
- The ideas carry over, so I can learn it fast

### Likely follow-up

**They might ask:** Why might this role prefer LangGraph?

**You say:** When you need tight, step-by-step control and want to save the agent's progress as it goes.

---

## A6 How do you test whether an agent system actually works? [EXPERT]

*Why they ask: Most people only check the final answer. They want to see you go deeper.*

### How to answer

I check two things: the result and the path. The result is simple: did the agent finish the task correctly on a set of test cases. The path is just as important: did it use the right tools, in a sensible number of steps, without wasting money. An agent that gets the right answer by luck through a messy path will fail next time, so I score the path too.

I track how often it succeeds, how often it picks the right tool, how many steps and how much it costs, and for search-based agents, whether the answer is backed by real sources. I keep a fixed set of test cases so I can tell if a change made things better or worse. My rule: I do not trust an agent based on a feeling. I measure it.

### Main points to make

- Check the result AND the path it took
- A lucky answer through a messy path is fragile
- Track success rate, tool choice, steps, and cost
- Keep a fixed test set to catch changes
- Measure it, do not trust a feeling

### Likely follow-up

**They might ask:** How many test cases do you need?

**You say:** Even 30 to 50 good ones catch most problems.

---

## A7 How does an agent remember things during a long task? [FOUNDATION]

*Why they ask: They want to check you understand the limits of an agent's memory.*

### How to answer

I split memory into two kinds. Short-term memory is the current task: the plan, recent results, and the last few tool outputs. I keep this in the active context. Long-term memory is anything that needs to last beyond this task. I store that outside, in a vector database or the knowledge graph, and pull it back only when needed.

Why? The agent's working memory is limited and gets expensive and less accurate when it is too full. So I summarise old steps, keep only what the next step needs, and fetch from long-term memory when relevant. In a team of agents, I keep the shared memory clear and easy to inspect so I can debug it.

### Main points to make

- Short-term memory vs long-term memory
- Long-term memory lives outside, in a database or graph
- Working memory is limited and degrades when too full
- Summarise old steps, keep only what is needed
- Keep shared memory clear and easy to inspect

### Likely follow-up

**They might ask:** When is the working memory too full?

**You say:** When the useful information gets buried under old, stale text. I trim or move it to long-term memory before that.

---

## B. GraphRAG

*Mixing a knowledge graph with vector search. This is the thing that sets you apart from people who only know basic search. Make the value of the graph clear.*

### B1 What is GraphRAG, and why mix a graph with vector search? [INTERMEDIATE]

*Why they ask: A basic question. They want to know you understand why the graph adds value.*

#### How to answer

First, in plain words. RAG means the AI looks up relevant text before answering, instead of guessing from memory. A vector search finds text that is similar in meaning. That is great for finding passages, but it does not understand how things connect. A knowledge graph is a map of things and how they link: this company owns that one, this person sits on that board. GraphRAG uses both together.

Why both? They answer different kinds of questions. Vector search is good for finding the right paragraph. The graph is good for following links between things. The case where plain vector search fails is a question that needs several hops, like which of our clients are linked to a supplier hit by some event. Similarity alone cannot connect those dots, because the answer lives in the links. The graph follows those links. So GraphRAG is not one or the other. It is both, each doing what it is best at.

#### Main points to make

- RAG = look up real text before answering
- Vector search finds similar text but misses links
- A graph is a map of things and how they connect
- Multi-step questions need the graph
- Use both together, each for what it is best at

#### Likely follow-up

**They might ask:** When is plain vector search enough?

**You say:** For simple fact look-ups and FAQs, where there are no links to follow.

---



## B2 Walk me through a GraphRAG pipeline from start to finish. [EXPERT]

*Why they ask: They want to know you can actually build it, not just define it.*

### How to answer

Step one: break the documents into small, meaningful pieces called chunks. Step two: read each chunk and pull out the things and their links to build the graph. Every thing and link points back to the chunk it came from, so we can prove where it came from. At the same time, I build a vector index over the chunks for similarity search. Then I group related things into clusters and write a short summary for each cluster, which helps with big-picture questions.

When a question comes in, I decide how to answer it. A specific question pulls the related part of the graph plus the closest chunks. A broad, big-picture question uses the cluster summaries instead. I combine what I found and write the answer with sources attached. The smart part is choosing the right path for each question and combining the results well.

### Main points to make

- Chunk the docs, then pull out things and links
- Link every thing back to its source chunk
- Build a vector search over the chunks too
- Group related things and summarise each group
- Route each question to the right path, then combine

### Likely follow-up

**They might ask:** How do you decide narrow vs broad?

**You say:** I look at the question. About one thing = use the graph. About overall themes = use the summaries.

---

## B3 How do you build the graph from raw documents? [EXPERT]

*Why they ask: This is the hardest part of GraphRAG. They want real experience.*

### How to answer

I break the documents into chunks first. Then I use an AI to read each chunk and pull out the things and their links in a fixed format. I limit the types of things and links to the ones we care about, ideally matching the ontology, so the graph stays clean instead of messy. Then comes the hardest part: matching duplicates. IBM and International Business Machines must become one thing, not two. This step decides how good the graph is.

After the graph is built, I group related things and summarise each group. One honest point: this reading step is not perfect, and the graph goes out of date as documents change. So I test the quality on a sample and plan to rebuild it now and then. I never treat the auto-built graph as perfect truth without checking.

### Main points to make

- Chunk first, then pull out things and links with AI
- Limit the types so the graph stays clean
- Matching duplicates is the make-or-break step
- Group and summarise after building
- Test the quality, expect it to go stale

### Likely follow-up

**They might ask:** Same thing, different names?

**You say:** I match duplicates using similarity plus some rules, and a human checks the unsure ones.

## B4 How do you test if your search is actually good? [EXPERT]

*Why they ask: This is your strong point. Make it count.*

### How to answer

This is something I am known for. I do not judge search quality by feel. I build a test set of questions where I already know the right sources. Then I measure: are the pieces I pulled actually relevant, did I find the pieces that hold the answer, and does every claim in the final answer come from a real source. That last one is the main control on the AI making things up.

For GraphRAG, I run a fair test: I compare the full graph-plus-vector setup against vector-only on the same questions. If the graph does not clearly improve the answers on the multi-step questions, it is not worth its cost and I remove it. This testing is the difference between a flashy demo and something you can trust. I have done this on a real pipeline.

### Main points to make

- Build a test set with known right answers
- Check relevance, coverage, and real sources
- Sources backing every claim controls made-up answers
- Compare graph+vector vs vector-only fairly
- If the graph does not help, cut it

### Likely follow-up

**They might ask:** What if the graph does not help in the test?

**You say:** Then I cut it. Extra cost with no proven benefit is not worth keeping.

---

## B5 How do you stop the AI from making things up? [INTERMEDIATE]

*Why they ask: This is critical for a company. They want to see your discipline here.*

### How to answer

Making things up is called hallucination, where the AI confidently says something untrue. The fix is built into the design, not just the prompt. Every claim in the answer must point back to a real chunk or a real thing in the graph. I keep the source on everything and ask the model to show its sources. If a claim has no source, it should not be in the answer, and a checking step removes it. The graph helps here because link-based claims can be checked against the real links.

Most made-up answers are really a search problem: the model invents because it was not given the right information. So better chunking and better search reduce made-up answers more than telling the model do not lie. For high-risk answers, a human still reviews.

### Main points to make

- Every claim must point to a real source
- Keep the source on everything, show it
- Drop any claim with no source
- Most made-up answers are really a search problem
- A human reviews high-risk answers

### Likely follow-up

**They might ask:** Is asking for sources enough?

**You say:** It helps, but I also check that the source really supports the claim.

---

## B6 What is the difference between narrow and broad questions in GraphRAG?

[INTERMEDIATE]

*Why they ask:* They want to know you understand the well-known GraphRAG pattern.

### How to answer

Narrow questions are about one thing, like what does this contract say about ending it. I answer by pulling that thing's nearby links and the matching text. Broad questions are about everything together, like what are the common risks across all these files. No single passage can answer that, so I prepare cluster summaries ahead of time and answer from those.

The point is that one search method cannot handle both. I send the question down the right path based on whether it is narrow or broad. That is what lets GraphRAG answer the full range of questions, not just narrow look-ups.

### Main points to make

- Narrow = about one thing. Use nearby graph + text.
- Broad = about everything. Use cluster summaries.
- No single passage answers a broad question
- Prepare cluster summaries ahead of time
- Route the question based on narrow or broad

### Likely follow-up

**They might ask:** How are cluster summaries made?

**You say:** I group related things in the graph, then summarise each group ahead of time.

---

## C. Context Engineering + MCP

*How you prepare and trim text for the AI, plus how you connect agents to data with MCP. Your strong point plus your rarest skill.*

### C1 How do you decide how to split documents into chunks? [INTERMEDIATE]

*Why they ask: Basic context engineering. They want to know you test it, not guess.*

#### How to answer

A chunk is a small piece of a document that the search looks through. I make each chunk a complete idea, like a paragraph or a section, instead of cutting at a fixed word count that might slice a sentence in half. I add a small overlap between chunks so a fact sitting on the boundary is not lost. Then I test different chunk sizes against my search test set instead of guessing.

I have done this for real. On one project I broke a large PDF into about 971 chunks and tuned the size based on how well search worked. I also hit a real bug while loading them into the database: it kept failing with EOF errors. I fixed it by saving them in small batches of 25 at a time. So I know chunking from both the design side and the messy real-world side.

#### Main points to make

- A chunk is one complete idea, not a fixed word count
- Add small overlap so boundary facts are not lost
- Test chunk sizes instead of guessing
- Real: 971 chunks, tuned by how well search worked
- Real bug: fixed loading errors with batches of 25

#### Likely follow-up

**They might ask:** Fixed size or smart chunks?

**You say:** Fixed size is simple but cuts ideas in half. Smart chunks follow the document and search better.

---

### C2 What extra labels do you add to chunks, and why? [INTERMEDIATE]

*Why they ask: They want depth beyond just chunking.*

#### How to answer

I attach extra labels to each chunk: where it came from, which section, which things it mentions, the date, and any sensitivity tags. Two reasons. First, proof: when the answer cites a chunk, I can trace it back. Second, filtering: labels let me search only the right kind of document, the right date range, or the right thing before similarity even runs, which makes results much sharper.

In a system with a knowledge graph, I match these labels to the graph's terms so the chunk speaks the same language as the graph. That match is what lets an agent move between the text and the graph without losing track.

#### Main points to make

- Labels: source, section, things, date, sensitivity
- Proof: trace the answer back to its source
- Filtering before search makes results sharper
- Match labels to the graph's terms
- This lets the agent move between text and graph

#### Likely follow-up

**They might ask:** Filter before or after search?

**You say:** Before, when I can. It shrinks the search space and improves results.

---

### C3 How do you handle a very large amount of text for the AI? [EXPERT]

*Why they ask: A must-have skill named in the job post.*

#### How to answer

When there is a lot of text to fit, the lazy option is to cut off whatever does not fit. That throws away things that might matter. Instead, I shrink the less important parts into short summaries so the meaning stays but uses far fewer words, and I keep full detail only for the parts that answer the question. For very long documents, I move a focus window across them in passes instead of forcing it all in at once.

The key idea: more text is not better text. Past a point, the useful information gets lost in the noise and quality drops. Models also pay less attention to the middle of a long block. So I search precisely, summarise the supporting bits, and put the most important text where the model reads it best. The goal is the right text, not the most text.

#### Main points to make

- Do not just cut off text that does not fit
- Summarise the less important parts to save space
- Keep full detail for the parts that matter
- More text is not better text
- Right text, not most text

#### Likely follow-up

**They might ask:** When does a bigger window not help?

**You say:** When the search is sloppy. A huge window full of weakly related text gives worse answers. Get search right first.

---

### C4 What is MCP and why does it matter? [INTERMEDIATE]

*Why they ask: The rarest skill on the list. They want real understanding, not a definition.*

#### How to answer

MCP, the Model Context Protocol, is a standard way to connect an AI to outside tools and data. Without it, you write custom code for every system you connect to. With MCP, a server offers its tools, its data, and its prompts in one common format, and any AI app can talk to it the same way. So instead of many custom connections, you get one clean, reusable contract, and one place to handle security and checks.

The deeper reason I like MCP is the boundary it creates. The AI does not get direct keys to your database. It only gets a safe, limited set of tools that the server decides. For a company's important data, that boundary is the whole point.

#### Main points to make

- MCP = a standard way to connect AI to tools and data
- One common format instead of custom code per system
- One place for security and checks
- The AI gets safe, limited tools, not direct keys
- The safety boundary is the real point

#### Likely follow-up

**They might ask:** How is MCP different from normal function calling?

**You say:** Function calling is one app letting the model call its own functions. MCP makes the server side standard, so any app can connect the same way, with the boundary built in.

---

## C5 Design an MCP server that lets agents query a knowledge graph. [EXPERT]

*Why they ask: Straight from the job post. They want your real MCP experience.*

### How to answer

The server is the safety boundary, and I design it carefully. I give the agent a small set of clear tools: one tool to query the graph that only accepts safe, checked queries, one tool to look up the catalog, and read-only data. The agent never gets direct database keys and cannot run anything it likes. Login, rate limits, and input checks all live on the server. Any writing to the data is a separate, locked-down tool.

I have built exactly this. My MCP server for Patient Connect Ops at Takeda runs on an EC2 server, sits behind a network load balancer, and uses a private connection, with the safety boundary on the server itself. That is why I see MCP first as a security and design problem. Connecting is easy. The discipline of a small, checked, trackable set of tools is what makes it safe for a regulated company.

### Main points to make

- The server is the safety boundary
- Give a small set of safe, checked tools
- No direct database keys for the agent
- Login, rate limits, and checks live on the server
- Real: EC2 + load balancer + private connection at Takeda

### Likely follow-up

**They might ask:** How do you stop a slow, expensive query?

**You say:** I limit the query tool: cap how deep it goes and how much it returns, and never allow open-ended queries.

---

## C6 Why use MCP instead of just a normal web API? [INTERMEDIATE]

*Why they ask: They want to know you understand what MCP adds over normal integration.*

### How to answer

You can put a normal web API in front of your systems, but then every AI app needs custom code to know it exists and how to use it. MCP makes this standard. An AI app can ask an MCP server what tools and data it has and use them, with no custom wiring. It is built for AI agents, so the tool descriptions come in a form the model can understand.

So a normal API is fine as the pipe underneath, but MCP is the standard contract on top, made for AI agents. The payoff is reuse: one server works across different AI frameworks, and the safety boundary is part of the standard instead of rebuilt every time.

### Main points to make

- A normal API needs custom code for every AI app
- MCP lets the app discover tools with no wiring
- Tool descriptions come in a form the AI understands
- One server works across different AI frameworks
- The safety boundary is built into the standard

### Likely follow-up

**They might ask:** Can MCP run on top of a normal API?

**You say:** Yes. The pipe can be normal web tech. MCP is the standard contract layer on top.

---

## D. Semantic Systems + Ontologies

*Your stretch area. Be honest, show you are close through GraphRAG, and treat deep ontology work as your partner's job. Do not bluff.*

### D1 Knowledge graph vs vector store. When do you use each? [INTERMEDIATE]

*Why they ask: They want to know you understand the role of the semantic layer.*

#### How to answer

A vector store keeps text as numbers and finds text that is similar in meaning. A knowledge graph keeps things and the clear links between them, and answers questions about how things connect. I use the graph when links and multi-step questions matter, when I need exact facts I can query, or when I want a fixed structure. I use the vector store when I am searching free text by meaning.

In a serious system I use both, which is the GraphRAG idea. The graph gives the backbone of links, the vector store gives broad text search. The graph and its ontology give an agent a clear map of the subject instead of just a pile of text.

#### Main points to make

- Vector store = find similar text
- Graph = things and the clear links between them
- Graph for links, exact facts, and structure
- Vectors for searching free text by meaning
- Serious systems use both: GraphRAG

#### Likely follow-up

**They might ask:** Can a vector store handle links?

**You say:** Only loosely and unreliably. Clear links belong in a graph where you can follow and check them.

## D2 What is an ontology, and why does it matter for agents? [EXPERT]

*Why they ask:* Your main stretch question. They want honesty and the right framing.

### How to answer

An ontology is a formal map of a subject: the types of things that exist, their traits, and how they relate, written clearly enough for a computer to reason about. In one common style (called RDF and OWL) it is stored as simple facts called triples and queried with a language called SPARQL. In another style (a property graph like Neo4j) it is nodes and links queried with a language called Cypher. The ontology is the shared dictionary that keeps everyone, and every agent, using the same words for the same things.

It matters because it gives agents a clear map to reason over, instead of guessing from free text. I will be straight about my level: I work comfortably with knowledge graphs, I understand the difference between the two styles and how agents query each, but deep ontology building is a craft I would partner with semantic data architects on. My job is the agent and MCP layer that uses the ontology, which is exactly how this role describes the teamwork.

### Main points to make

- Ontology = the types of things, their traits, and links
- RDF/OWL stores facts as triples, queried with SPARQL
- Neo4j stores nodes and links, queried with Cypher
- It is the shared dictionary for everyone and every agent
- Honest: I use ontologies, I partner on building them

### Likely follow-up

**They might ask:** Triple store vs property graph?

**You say:** RDF/OWL is standards-based with strong formal reasoning. Neo4j is more developer-friendly. The choice depends on whether you need formal reasoning or easy traversal.

---

## D3 How do you make a graph easy for agents to search? [INTERMEDIATE]

*Why they ask:* Pulled straight from the job post.

### How to answer

From the agent's side, two things help most: short, clearly-labelled paths and good names. If a common question needs six hops through the graph, it is slow and easy to get wrong, so I would work with the semantic data architects to add direct links or shortcuts for the common cases. Clear, consistent link names matter because the agent reasons about what each link means. Good plain-English labels help the AI match a question to the right part of the graph.

I would also let the agent reach the graph through a small set of safe MCP tools rather than open access, so it follows good paths instead of slow ones. The teamwork is clear: the architects own that the ontology is correct, and I tell them which shapes make agent search work well.

### Main points to make

- Short, clearly-labelled paths help most
- Add shortcuts for the common questions
- Clear, consistent link names so the agent gets meaning
- Reach the graph through safe MCP tools, not open access
- Architects own correctness, I feed back what works

### Likely follow-up

**They might ask:** How do you find the common paths?

**You say:** From the agent's query logs. The searches it runs most are the ones worth speeding up.



## D4 How would you auto-tag incoming data against an ontology? [INTERMEDIATE]

*Why they ask: A named job in the post. They want your NLP-plus-AI approach.*

### How to answer

I use two methods together and then match to the ontology. A classic NLP method picks out the obvious things. An AI method handles the trickier cases that simple rules miss. The key step: I do not let the model invent its own labels. I give it the relevant part of the ontology and tell it to map to those real terms only, with a few examples to lock the format.

Then I add a confidence score to each tag. High-confidence tags go through automatically. Low-confidence ones get sent to a person to review, instead of quietly adding wrong data. And I test the tagging accuracy on a sample before trusting it, because a tagger that is wrong even one time in ten slowly corrupts everything later.

### Main points to make

- Use classic NLP plus AI together
- Do not let the model invent its own labels
- Map to real ontology terms, with examples
- Add a confidence score per tag
- High confidence auto-flows, low confidence gets reviewed

### Likely follow-up

**They might ask:** Why not let the AI tag freely?

**You say:** It drifts and makes up labels. Tying it to the ontology and checking it keeps the data consistent.

---

## D5 What is a data catalog and why does an agent need one? [FOUNDATION]

*Why they ask: The job post mentions metadata catalogs. They want breadth.*

### How to answer

A data catalog is the master list of what data exists, where it is, and what it means. It is the map. For an agent it is essential, because the agent has to know what data exists before it can use it. Without a catalog, the agent is searching blind.

In the agent layer, I would let the agent reach the catalog through an MCP tool so it can look up what things and datasets exist, then decide what to fetch. The catalog plus the ontology give the agent both the list of what is there and the meaning of it. That is the base for trustworthy answers.

### Main points to make

- A catalog is the master list of what data exists
- It is the agent's map
- An agent must know what exists before it can use it
- Reach the catalog through an MCP tool
- Catalog + ontology = the list plus the meaning

### Likely follow-up

**They might ask:** How does the catalog link to the graph?

**You say:** The catalog points to the things the ontology defines and the graph fills in. They are layers of the same system.

---

## E. Behavioral / STAR

*Real stories from your work. Use four parts: Situation, Task, Action (most of it), Result. Keep each under three minutes. Plain, confident words.*

### E1 Tell me about a complex AI system you built from start to finish. [INTERMEDIATE]

*Why they ask: Your anchor story. They want real, hands-on depth.*

#### How to answer

**Situation:** At Takeda, teams spent a lot of time digging through CI/CD logs to figure out pipeline problems, and the know-how was scattered across people. **Task:** I wanted to build something that could answer their questions directly, instead of making people search through logs by hand.

**Action:** I built a DevOps Intelligence Platform in three parts. One, an MCP server on an EC2 box behind a load balancer with a private connection and a safety boundary, so agents had a safe way in. Two, a search (RAG) pipeline that read our docs, chunked them, and gave answers backed by real sources. Three, an agent layer that could reason about the pipeline. I used LangChain and ChromaDB for the search part and AutoGen for the agents. **Result:** It turned a slow, manual, in-people's-heads process into something you can just ask. That design, MCP plus search plus agents, is now my go-to pattern for exactly this kind of role.

#### Main points to make

- Built a DevOps platform: MCP + RAG + agents
- MCP server on EC2 with a safety boundary
- LangChain and ChromaDB for search, AutoGen for agents
- Turned scattered know-how into something you can ask
- It is my go-to pattern for this role

#### Likely follow-up

**They might ask:** Hardest part?

**You say:** Making the answers trustworthy and getting the data loaded cleanly. I solved both by testing and adjusting, not guessing.

## E2 Tell me about a decision you made without full information. [INTERMEDIATE]

*Why they ask: They want to see good judgement under uncertainty.*

### How to answer

Situation: While building the search pipeline, loading a big batch of chunks (about 971) into the database kept failing with errors. Task: I needed loading to work reliably, but I did not yet know the exact cause and did not have endless time to dig.

Action: Instead of stalling, I made a safe, easy-to-undo decision. I guessed the problem was the large batch size, so I changed it to load in small batches of 25 at a time. Low risk, easy to reverse, and a clear pass-or-fail signal. Result: The errors stopped and loading became reliable. The lesson I carry: when you do not have full information, make the smallest safe change that gives a clear answer, rather than waiting for perfect certainty.

### Main points to make

- Real: loading 971 chunks kept failing
- Did not stall, made a safe, reversible change
- Guessed it was batch size, switched to batches of 25
- It worked and loading became reliable
- Lesson: smallest safe change over waiting for certainty

### Likely follow-up

**They might ask:** Did you find the real root cause later?

**You say:** The fix held and loading stayed stable, so I moved on. I would dig deeper if it came back at bigger scale.

---

## E3 Tell me about getting people to use something you built, without authority. [EXPERT]

*Why they ask: They want to see influence, which matters for a POC role.*

### How to answer

Situation: I built two CI/CD tools, an AI CI/CD Guardian and an Action Log Analyser, and published them on the GitHub Marketplace under my SKCloudOps name. Task: Tools are useless if no one uses them, and I had no power to force anyone.

Action: I led with the problem, not the tool. I showed people the real pain: noisy failing pipelines and slow log checking. Then I made my tools the easy choice by packaging them to fit straight into people's existing workflow with almost no effort. I wrote clear docs and improved them based on feedback. Result: They became public proof of the exact agent-plus-DevOps thinking this kind of role wants. The lesson: solve a real pain and remove friction, do not force people. That is how I would get a POC adopted here too.

### Main points to make

- Built and published two real tools, no authority
- Led with the problem, not the tool
- Made my tool the easy choice, low effort to adopt
- Clear docs, improved on feedback
- Lesson: solve real pain and remove friction

### Likely follow-up

**They might ask:** What if someone resists?

**You say:** I find their real objection, usually it is effort or trust, and fix that instead of pushing harder.

---

## E4 How do you work with a team that owns something you depend on? [INTERMEDIATE]

*Why they ask: This matches working with the Semantic Data Architects in the post.*

### How to answer

I treat it as a clear handoff with respect for who owns what. In this role, the semantic data architects own that the ontology is correct and well-structured. I own the agent and MCP layer that uses it. To make this work, I am specific about what I need from them, the link shapes and labels that make agent search reliable, and I give them clear feedback, like which searches my agents run most and where they fail, so they can improve it with real data.

I have done this kind of cross-team work for years at Takeda, sitting between the platform, app, and operations teams. The pattern that works: agree on a clear handoff, speak in their terms, and bring facts not opinions. I do not try to take over their job. I just make the handoff between our two areas clean.

### Main points to make

- Clear handoff, respect who owns what
- They own the ontology, I own the agent layer
- Be specific about what I need from them
- Give feedback with real data, not opinions
- Do not take over their job, keep the handoff clean

### Likely follow-up

**They might ask:** What if their structure does not work for your agents?

**You say:** I bring the exact failing cases and the data, and we fix it together.

---

## E5 How do you handle a POC with a tight deadline? [FOUNDATION]

*Why they ask: This role is POC-level, so they want to see how you scope.*

### How to answer

I focus hard on the riskiest unknown first. A POC exists to answer one question, usually can this approach actually work for us. So I find the one thing most likely to sink the idea and test that before polishing anything. For an agent-over-graph POC, the risky part is usually whether the search and reasoning are good enough, so I get a thin, end-to-end version working on real data fast, then measure it. I do not build lots of features that do not reduce the risk.

I am also clear with everyone about what the POC will and will not show, because a quick demo, a proven accuracy number, and a path to production are very different goals. And I design the thin version so the parts that work can grow into production later. Because I have built production systems, I know what the prototype will eventually need, so I avoid early choices that would trap us.

### Main points to make

- Test the riskiest unknown first
- Get a thin end-to-end version working fast, then measure
- Do not build features that do not reduce risk
- Be clear what the POC will and will not show
- Build so it can grow into production later

### Likely follow-up

**They might ask:** When is a POC finished?

**You say:** When it has answered its question with evidence, yes or no. Not when it looks polished.

---

# Worked examples

Open-ended design questions. Two are named in the job post. Talk through the steps out loud and draw the boxes.

## 1. Credit risk agent (named in the job post)

*"Build an agent that helps an analyst judge the credit risk of a company client, using our knowledge graph of companies, filings, and links."*

### HOW I FRAME IT

This needs several steps of linked reasoning, which is exactly where GraphRAG shines. Credit risk is not one document. It is a web: the client, its sub-companies, its guarantors, shared directors, its industry, and past filings. Plain text search loses those links. So I use the graph to follow the links and vector search to read the filings and news.

### THE DESIGN

A planner agent splits the request into steps: find the companies, map the exposure, pull the financial signals, then write the risk summary. Each step is a tool call. Graph tools answer who is linked to whom, behind an MCP server. A search tool pulls the right text from filings and news. A checker agent makes sure every claim in the summary points back to a real source.

### CONTEXT HANDLING

I attach the source labels to every chunk so the model never uses an unsourced fact. For long filings I summarise the less important parts instead of cutting them off. The risk rules go in a fixed system setting, not the user's question.

### TRADE-OFFS TO MENTION

Freedom vs trust. In credit risk the answer must be defensible, so I limit how freely the agent acts, require a source trail, keep a human approving the final risk grade in a POC, and judge the agent on whether its evidence is right, not just the grade.

---

## 2. Customer onboarding workflow (named in the job post)

*"Build an agent workflow that takes a new customer's data, tags it against our ontology, matches it to existing records, and flags conflicts."*

### HOW I FRAME IT

This is auto-tagging plus matching new data against what we already have, which the role mentions directly. The agent's job is to take messy incoming data and make it fit the company's shared dictionary (the ontology).

### THE DESIGN

An intake agent cleans up the new records. A tagging agent reads them and maps the terms to real ontology types, not free text. A matching agent searches the graph for likely matches and scores them. A conflict agent flags the unsure matches for a person, instead of quietly merging them. MCP sits between the agents and the graph so they get a safe, controlled way in.

### CONTEXT HANDLING

I give the tagging agent the relevant slice of the ontology so it uses real terms. A few examples lock the format.

### TRADE-OFFS TO MENTION

Accuracy vs speed. Auto-merging everything is fast but ruins the data on a bad match. So I flag anything below a confidence line for review, and let the business set that line. For a POC, I prove the tagging and matching accuracy on a sample before talking about scale.

### 3. GraphRAG pipeline over many documents

*"We have thousands of internal documents. Build a GraphRAG pipeline that answers both narrow and broad questions."*

#### WALK THE STEPS

Break documents into meaningful chunks. Read them to pull out things and links and build the graph. Group related things and summarise each group for big-picture questions. Build a vector search over the chunks too. When a question comes in, send narrow ones to the graph plus matching chunks, and broad ones to the group summaries. Combine and answer with sources.

#### TESTING

This is where I stand out. I do not ship search on a feeling. I measure how relevant and faithful the answers are on a known test set, and I compare graph-plus-vector against vector-only to prove the graph is helping. If it is not, I cut it.

#### TRADE-OFFS TO MENTION

Building the graph costs time and money, and it goes stale as documents change. I say that openly and only recommend the graph where multi-step questions need it. For a simple FAQ, plain vector search is right, and I will say so.

---

### 4. MCP server for safe access to the graph

*"How would you build an MCP server so agents can safely query our knowledge graph and catalog?"*

#### THE DESIGN

The server is the safety boundary. Agents never get direct database keys. I give them a few clear tools: a safe, limited query tool, a catalog look-up, and read-only data. Small and clear, so the agent cannot run anything it likes or change the data. Login and rate limits live on the server.

#### GROUNDED IN MY WORK

I have built this. My MCP server for Patient Connect Ops at Takeda runs on EC2, behind a network load balancer, with a private connection and the safety boundary on the server. That is why I think about MCP as a security and design job first.

#### TRADE-OFFS TO MENTION

A tighter set of tools is safer but less flexible. For a regulated company, that is the right default. I would rather add a tool on purpose than leave an open query door and hope the prompt holds.

---

# Pitch and extras

## Your 90-second opening

"I am a Cloud and AI platform architect with over fifteen years of experience. Right now I build the behind-the-scenes layer that makes AI systems work, at Takeda. My focus is the part most people skip: not just calling an AI, but building the structure that makes AI agents reliable. At Takeda I built a DevOps Intelligence Platform with three parts: an MCP server, a search (RAG) pipeline, and an agent layer. The MCP server runs on a real cloud setup with a proper safety boundary. The search pipeline turns a big set of documents into answers backed by real sources. And I have hands-on experience with GraphRAG, LlamaIndex, and AutoGen. What I bring to this role is exactly the stack in your job post: agent workflows, graph-plus-vector search, context engineering, and MCP. I am happy to own the agent layer and work with your semantic data architects on the ontology side. I am keen on this role because it is the agent-over-knowledge-graph problem, which I think is where the real value in company AI is going."

## What to say about your weak spots

### If they ask about LangGraph

"My hands-on work has been with LangChain, LlamaIndex, and AutoGen. LangGraph is the graph-based version of LangChain for step-by-step agent flows, and it matches the planner-and-worker setup I already build in AutoGen. I have studied how it works, and I am confident I can be productive in it quickly. The ideas carry over, the tool is the easy part."

### If they push on ontologies, RDF, OWL, SPARQL

"I will be straight with you. I am the agent-layer architect, not a full-time ontology expert. I am comfortable working with knowledge graphs, I know the difference between the two main graph styles and how agents query each, and I know how to build the agent and MCP layer that uses an ontology. The deep ontology building is something I would partner with your semantic data architects on, which is how your job post describes it too. My value is making the semantic layer usable by agents."

### Simple rule for weak spots

Say the gap honestly in one sentence, connect it to a nearby strength, then stop. Do not over-apologise, and do not bluff. Interviewers trust I would partner on that far more than a fake expert.

## Good questions to ask them

- How built-out is the semantic layer today, and who owns the ontology?
- What does POC success mean here: a demo, a proven accuracy number, or a path to production?
- When a POC works here, what happens to it? Does another team take it to production, or does this team carry it forward? (This tells you how stable the role is.)
- Is the knowledge graph the RDF style or the property style, and how is it queried today?
- Are you standardising on MCP across teams, or is this fresh ground?
- How do the agent architect and the semantic data architects split the work day to day?

## Night-before checklist

### Have a clear one-minute version ready of:

your MCP server and why the safety boundary matters; your GraphRAG pipeline and how you tested it; your chunking and the 971-chunk story; and the credit-risk and onboarding examples.

**Easy lines to remember:**

graph plus vector together; every answer points back to a real source; the MCP server is the safety boundary; I measure search, I do not ship on a feeling; I am the agent layer, I partner on the ontology.

**Tone:**

lead with clear thinking and design. POC mindset. Honest about gaps. Plain words, no jargon soup. You are very strong on the agent, MCP, and context parts. Walk in knowing that.